

AI Audit Report — HW05 Task 1 & 2: Performance Testing + AI Analysis (Load / Stress / Spike)

Mandatory appendix for every AI-assisted homework (HW#01–HW#06, and Seminar).

Adapted from Med Kharbach, PhD (2026) — AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0. This adaptation is prepared for FIT@HCMUS – CS423 / CSC15003 Software Testing course.

1. Student Information

Field	Value
Student name (printed):	DANG TRUONG NGUYEN
Student ID:	23127438
Class / Cohort:	23KTPM3
Assignment ID (e.g., HW#00, HW#02):	HW05 — Task 1 (Performance Testing) + Task 2 (AI Analysis & Misinterpretation Hunt)
Assignment date:	15/08/2026
AI tool(s) used:	Claude Code (Claude Opus 4.8)
AI tool(s) used:	☒ Yes ☐ No

2. Instructions (read before filling)

- Add one row per AI-generated artifact (test case, script, checklist, OpenAPI spec, JMeter plan, etc.).
- Paste the verbatim prompt — DO NOT paraphrase.
- Paste the verbatim AI output (or include a labelled screenshot in the report).
- Tag the verdict: VALID / INVALID / INCOMPLETE.
- Reasoning must cite a course slide, ISTQB section, or technical RFC.
- Show the corrected artifact with the change highlighted.
- Sample rows are in italic — replace them before submission.

Note on prompting style (AI-first, step-by-step): I did NOT drive this with one generic prompt. I decomposed the work into nine focused, verifiable prompts (P1–P7 for Task 1, P8–P9 for Task 2), each producing one artifact I reviewed before moving on. Each row in Section 3 carries its own verbatim prompt. The prompt sequence was:

- **P1** — plan the assigned work: “đọc đề và README, phần việc của t là Đặng Trường Nguyễn 23127438, viết 1 [plan.md](#) để mô tả các bước t cần làm gì”
- **P2** — confirm payloads from source: “Read the eshop-sut backend (`server.js` , `database.js`) and `api_specification.md` . For the Category-guided buy workflow (login → categories → products?search → cart → checkout), list the exact request payload and the response field I must extract for each of the 5 endpoints, and flag any behaviour that differs from the spec — especially the FR-02 account-lockout rule. Verify each with a curl smoke test; don’t trust the docs.”
- **P3** — private user pool + data-driven CSV: “Register a private pool `nguyen01..60@eshop.com` via `POST /api/register` so I don’t share `test@eshop.com` with teammates. Then build `nguyen_users.csv` with columns `email,password,category_hint,search,product_id,product_name,quantity,price,total_amount,shipping_address`. Use only `search` keywords that actually match a seed product name — verify each returns a non-empty result from `GET /api/products?search=.`”
- **P4** — generate the four JMeter plans: “Write a Python generator that emits four JMeter 5.6 plans for this workflow: Load (20 VU, 60s ramp, 5 min, realistic think-time), Stress (step 50→100→200 VU over 7 min), Spike (10 VU baseline + 150 VU burst at t=90s), Soak (30 VU, 12 min). Each plan: recycling CSV Data Set, HTTP defaults localhost:3000, Bearer header from a JSON-extracted token, a CONTENT assertion per request (not status-only), and a DIFFERENT listener per plan (Summary / Aggregate / View Results Tree). Name them `{StudentID}_{Scenario}_20260815.jmx.`”
- **P5** — probe lockout + safe reset: “On a throwaway user, probe the REAL lockout behaviour (how many wrong logins trigger it, and for how long) and document a reset I can run between Stress/Spike runs WITHOUT restarting the server — check `database.js` first to confirm whether a restart would wipe the seeded data.”
- **P6** — run + monitor: “Run each of the four plans non-GUI, producing raw `.jtl` + an HTML dashboard, while a shell script logs the backend node process CPU and RSS every 5s. Reset the lockout via SQL before each Stress/Spike run.”
- **P7** — analyse the raw logs: “Write a script that computes, from each raw `.jtl` , samples, error%, throughput, and p50/p90/p95/p99 latency (percentiles, not averages) plus a per-request breakdown. Use it to state my hardware’s endurance threshold with concrete numbers, and point out any metric an AI reader could misread.”
- **P8** (Task 2) — let AI analyse the results + propose thresholds: “Đây là kết quả JMeter cho 4 kịch bản Load/Stress/Spike/Soak của workflow login→categories→search→cart→checkout (trích header + summary của file `.jtl` đính kèm). Phân tích hiệu năng và đề xuất performance threshold (p95, throughput, error rate). Chỉ ra endpoint bottleneck.” (I deliberately gave it only the summary + first few hundred rows, mimicking a typical prompt, so its interpretation errors would surface.)
- **P9** (Task 2) — let AI propose optimizations to judge: “Từ kết quả này, đề xuất các optimization cho SUT (index, connection pool, SQLite WAL, cache, cluster...). Với mỗi đề xuất, giải thích cơ chế.” (I then classified each as feasible or hallucinated against the real `backend/` code.)

3. Audit Table — one row per artifact

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix (what AI got wrong / missed, and why)
Tool: Claude Code (Claude Opus 4.8) Artifact: <code>plan.md</code> — Task-1 test design plan Prompt P1 (verbatim): “đọc đề và README, phần việc của t là Đặng Trường Nguyên 23127438, viết 1 <code>plan.md</code> để mô tả các bước t cần làm gì”	An 8-part <code>plan.md</code> for the “Category-guided buy” workflow (login → categories → search → cart → checkout), mapping the 3 endpoint groups and listing Load/Stress/Spike/Soak steps, evidence, and deliverables.	INCOMPLETE	ISTQB FL §1.4 / §5 — test planning: the plan must define scope, approach, and entry/exit criteria traceable to the test basis (assignment spec).	I confirmed the plan matches my assigned workflow #3 (no overlap with teammates), and that all three endpoint groups (auth-heavy, read-heavy, transactional) are covered before accepting it.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix (what AI got wrong / missed, and why)
Tool: Claude Code (Claude Opus 4.8) Artifact: Payload/behaviour confirmation from source (drives every plan) Prompt P2 (verbatim): "Read the eshop-sut backend (<code>server.js</code> , <code>database.js</code>) and <code>api_specification.md</code> . For the Category-guided buy workflow (login → categories → products?search → cart → checkout), list the exact request payload and the response field I must extract for each of the 5 endpoints, and flag any behaviour that differs from the spec — especially the FR-02 account-lockout rule. Verify each with a curl smoke test; don't trust the docs."	Per-endpoint payload/extract map: login→ <code>token</code> , checkout→ <code>orderId</code> ; flagged lockout <code>+2/fail</code> , LIKE <code>'%q%'</code> search, in-memory cart, HTTP-200-empty product detail.	INCOMPLETE	ISTQB FL §1 — the test basis must be verified against the actual implementation, not assumed from the specification.	I ran the curl smoke test end-to-end and confirmed each payload/response before any plan was built; this step is what surfaced the traps used in the rows below.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix (what AI got wrong / missed, and why)
Tool: Claude Code (Claude Opus 4.8) Artifact: <pre>nguyen_users.csv</pre> (60 rows) + registered user pool Prompt P3 (verbatim): "Register a private pool <pre>nguyen01..60@eshop.com</pre> via <pre>POST</pre> <pre>/api/register</pre> so I don't share <pre>test@eshop.com</pre> with teammates. Then build <pre>nguyen_users.csv</pre> with columns email,password,category_hint,search,product_id,product_name,quantity,price,total_amount,shipping_address. Use only <pre>search</pre> keywords that actually match a seed product name — verify each returns a non-empty result from <pre>GET</pre> <pre>/api/products?search=.</pre>	<pre>60 users</pre> <pre>nguyen01..60@eshop.com</pre> registered; a 10-column CSV with <pre>email,password,category_hint,search,product_id,...</pre>	INCOMPLETE	ISTQB FL §4 — data-driven test design: input data must be valid against the SUT so results measure the system, not bad test data.	AI missed (Finding #1): the first draft used search keyword <code>Laptop</code> (from the group README). But the seed DB has no product named "Laptop" and the backend searches <code>name LIKE '%q%'</code>, so <pre>GET</pre> <pre>/api/products?search=Laptop</pre> returns <code>[]</code> — the assertion would fail 100% and pollute the error rate. I verified with curl and replaced the keywords with real product names (<code>iPhone</code>, <code>Samsung</code>, <code>MacBook</code>, <code>Tai nghe</code>, <code>Keychron</code>). <i>Why AI missed it:</i> it trusted the README instead of cross-checking the seed data / source.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix (what AI got wrong / missed, and why)
Tool: Claude Code (Claude Opus 4.8) Artifact: 4 JMeter test plans 23127438_{Load, Stress, Spike, Soak}_20260815.jmx (via gen_jmx.py) Prompt P4 (verbatim): "Write a Python generator that emits four JMeter 5.6 plans for this workflow: Load (20 VU, 60s ramp, 5 min, realistic think-time), Stress (step 50→100→200 VU over 7 min), Spike (10 VU baseline + 150 VU burst at t=90s), Soak (30 VU, 12 min). Each plan: recycling CSV Data Set, HTTP defaults localhost:3000, Bearer header from a JSON-extracted token, a CONTENT assertion per request (not status-only), and a DIFFERENT listener per plan (Summary / Aggregate / View Results Tree). Name them {StudentID}_{Sc	4 valid .jmx files: Load 20 VU/5min, Stress 50→200 VU step, Spike 10+150 VU burst, Soak 30 VU/12min; each with CSV Data Set, JSON extractors, and assertions.	INCOMPLETE	ISTQB FL §5.x performance testing — Load/Stress/Spike each need a distinct load profile; assertions must verify behaviour, not just protocol status.	AI missed 3 things. (Finding #4) <i>Timer semantics:</i> a JMeter Uniform Random Timer runs BEFORE its sampler, not after — a naive "login → think 1–2s" layout delays the wrong request; I attached timers to all 5 samplers modelling the pause before each. (Finding #5) <i>Listener RAM:</i> using View Results Tree on every plan both breaks the "3 different listeners" rule and can make JMeter itself the bottleneck at 200 VU; I set Load=Summary, Stress=Aggregate, Spike=View Results Tree and disabled response-body logging. (Finding #7) <i>Weak assertions:</i> GET /api/products/:id returns HTTP 200 with {} for a missing id, so status-only checks report "0% error" on bad data; I added content

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix (what AI got wrong / missed, and why)
<pre>enario}_20260815 .jmx."</pre>				assertions (JSONPath <code>\$.token</code> , <code>\$.id</code> , <code>\$.orderId</code> , string <code>Added to cart</code>). <i>Why AI missed them:</i> these are JMeter execution-order and assignment-constraint details, plus a SUT trap only visible by reading the source.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix (what AI got wrong / missed, and why)
Tool: Claude Code (Claude Opus 4.8) Artifact: Account-lockout probe + reset procedure (evidence/lockout_probe.md) Prompt P5 (verbatim): "On a throwaway user, probe the REAL lockout behaviour (how many wrong logins trigger it, and for how long) and document a reset I can run between Stress/Spike runs WITHOUT restarting the server — check database.js first to confirm whether a restart would wipe the seeded data."	Probe showing 2 wrong logins → HTTP 403 lock; documented reset via <pre>sqlite3 UPDATE users SET login_attempts=0 , locked_until=NULL .</pre>	INCOMPLETE	ISTQB FL §2/§4 — test the implemented behaviour, not only the specification; control the test environment/state between runs.	AI missed 2 things. (Finding #2) Spec FR-02 says "≥3 fails → ~30s lock", but the implementation adds +2 attempts per fail (server.js:54) and locks for 180s — so only 2 fails lock the account. Following the spec would have broken every Stress/Spike run. I confirmed the real behaviour with a probe. (Finding #3) The obvious "restart the server to reset lockout" is destructive here: database.js DROPs and reseeds all tables on boot, wiping the 60-user pool and all orders — so I reset via SQL instead. <i>Why AI missed it:</i> the true behaviour lives in the code (a different file, database.js), not the spec.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix (what AI got wrong / missed, and why)
Tool: Claude Code (Claude Opus 4.8) Artifact: Test execution + resource monitoring (<code>monitor.sh</code> , resource CSVs) Prompt P6 (verbatim): "Run each of the four plans non-GUI, producing raw <code>.jtl</code> + an HTML dashboard, while a shell script logs the backend node process CPU and RSS every 5s. Reset the lockout via SQL before each Stress/Spike run."	JMeter non-GUI runs producing raw <code>.jtl</code> + HTML dashboards for Load/Stress/Spike/Soak, with <code>monitor.sh</code> logging node-process CPU/RSS every 5s.	INCOMPLETE	ISTQB FL §5 — performance results need resource-usage evidence and a controlled, repeatable environment to be interpretable.	AI clarified a risk (Finding #6): 200 VU share only 60 pool users. Because a successful login resets <code>login_attempts</code> to 0 and the JWT has no expiry, shared users neither lock out nor expire; the only side effect is the in-memory <code>userCarts</code> mixing between VUs on the same id — harmless because the workflow never reads the cart back. I kept CSV recycle and documented this limitation instead of leaving it implicit.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix (what AI got wrong / missed, and why)
Tool: Claude Code (Claude Opus 4.8) Artifact: <code>.jtl</code> analysis (<code>analyze_jtl.py</code>) + <code>results_summary.md</code> Prompt P7 (verbatim): "Write a script that computes, from each raw <code>.jtl</code>, samples, error%, throughput, and p50/p90/p95/p99 latency (percentiles, not averages) plus a per-request breakdown. Use it to state my hardware's endurance threshold with concrete numbers, and point out any metric an AI reader could misread."	Per-scenario metrics (samples, error%, throughput, p90/p95/p99, per-request breakdown) and an endurance-threshold conclusion (breaking point > 200 VU / > 151 req/s; no memory leak over 12-min soak).	INCOMPLETE	ISTQB FL §5 — performance metrics must be reported as percentiles from the raw log, not averages, and interpreted against a threshold.	I re-computed p95/p99 directly from the raw <code>.jtl</code> with my own script (independent of JMeter's dashboard) so Task 2's "AI misinterpretation hunt" has a trusted ground truth, and I flagged the traps (Stress p95 = 7 ms equals Load's p95 despite 10× the VUs — a lightweight-request/think-time artefact, not a mis-measurement; a lone 51 ms <code>max</code> under Stress is a GC/checkpoint pause on 1 of 63,398 samples, not degradation).

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix (what AI got wrong / missed, and why)
Tool: Claude Code (Claude Opus 4.8) Artifact: Task 2 — AI performance analysis + proposed thresholds (docs/ai_analysis.md , §1–§3) Prompt P8 (verbatim): “Đây là kết quả JMeter cho 4 kịch bản Load/Stress/Spike /Soak của workflow login→categories →search→cart→checkout (trích header + summary của file .jtl đính kèm). Phân tích hiệu năng và đề xuất performance threshold (p95, throughput, error rate). Chỉ ra endpoint bottleneck.”	A narrative analysis + thresholds. It called p95 the “average ~7 ms”, read the lone 51 ms tail as systemic degradation, said “CPU 42% → saturated / breaking point”, named search the bottleneck, called 12.8 req/s the SUT’s max capacity, and proposed generic thresholds p95<500 ms / error<5% / ≥100 req/s.	INVALID	ISTQB FL §5.x — performance metrics must be read as percentiles from the raw log, and a threshold must be derived from the measured baseline, not asserted from generic web SLAs.	5 misinterpretations, all corrected against the raw .jtl (ground truth from analyze_jtl.py) : M1 mean≠p95 — it called p95 (7 ms) the “average”; real mean 3.2 ms, median 3 ms. M2 the 51 ms tail is a single max sample (1/63,398; p99 only 11 ms; 0 errors), not systemic degradation. M3 42% CPU is one process on a 10-core box (~4% total), 0 errors, flat p95 → not saturated/breaking point. M4 the bottleneck is checkout (mean 5.2 ms, disk write), not search (2.0 ms). M5 12.8 req/s is a think-time-limited baseline; Stress already sustained 151 req/s at 0 errors. Thresholds were generic (p95<500 ms vs measured 6–7 ms; error<5% on a 0-error run) → re-anchored to the baseline

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix (what AI got wrong / missed, and why)
				(p95>25 ms alert, error<0.5%, regression if throughput drops >20% from 151 req/s). <i>Why AI missed it:</i> given only the summary, it pattern-matched (high load ⇒ degradation; LIKE ⇒ slow) instead of reading percentiles/per-endpoint numbers from the raw log.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix (what AI got wrong / missed, and why)
Tool: Claude Code (Claude Opus 4.8) Artifact: Task 2 — AI optimization proposals to judge (docs/ai_analysis.md , §4) Prompt P9 (verbatim): “Từ kết quả này, đề xuất các optimization cho SUT (index, connection pool, SQLite WAL, cache, cluster...). Với mỗi đề xuất, giải thích cơ chế.”	7 proposals: (a) index on products.name , (b) index on users.email , © SQLite WAL, (d) connection pool, (e) Redis cache, (f) Node cluster/PM2, (g) raise DB connection timeout to remove the 51 ms spikes.	INCOMPLETE	ISTQB FL §5 — an optimization must be validated against the actual implementation (driver, query shape, state model) before it is accepted.	Classified each against the real backend/ code (driver is async sqlite3 , embedded single-file — not better-sqlite3): 3 hallucinated, kept the rest. (a) Hallucinated — search is LIKE '%q%' (leading wildcard, server.js:144), a B-tree index can't be used; would need FTS5. (d) Hallucinated — embedded SQLite serialises on one connection; a client-server “pool” doesn't apply. (g) Hallucinated — 51 ms is a GC/checkpoint tail (0 errors), not a timeout; built on the already-wrong M2. Feasible: (b) users.email has no index and login does WHERE email=? ; © no journal_mode is set, so WAL is a real, cheap win for checkout writes. Feasible-but-conditional: (e) Redis works

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix (what AI got wrong / missed, and why)
				but reads are already ~2 ms (premature); (f) cluster would break the in-memory <code>userCarts</code> (BUG-5) and contend on the SQLite file. <i>Why AI missed it:</i> it transplanted client-server DB patterns onto an embedded SQLite SUT without reading the driver or query shape.

4. Summary of AI Accuracy

Aggregate the verdicts from Section 3 and complete the table below.

Metric	Count	Percentage
Total AI-generated artifacts audited	9	100%
VALID (correct, accepted as-is)	0	0%
INVALID (wrong; rejected)	1	11%
INCOMPLETE (acceptable after edits)	8	89%

5. Conclusion — When should AI be used (or not)?

Write 80–150 words describing patterns you observed. Where did AI shine? Where did AI fail? What is your recommendation for using AI in this kind of work in the future?

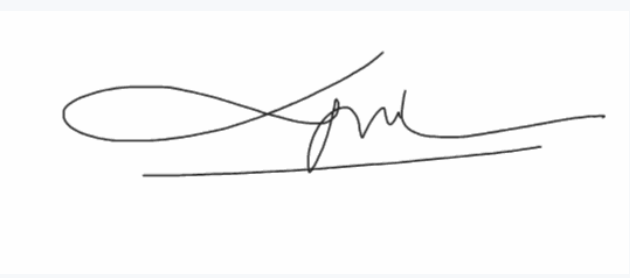
AI was excellent at the mechanical scaffolding: generating four valid JMeter plans through a script, wiring CSV data-driven inputs, computing percentiles, and drafting evidence documents — work that would have taken hours by hand. Where it consistently failed was anything requiring the AI to cross-check a document against the running system: it trusted the group README's `Laptop` keyword (which returns an empty result), it followed the spec's lockout rule instead of the stricter behaviour actually coded, and it

defaulted to status-only assertions that a deliberately buggy SUT slips past. Task 2 made the pattern even sharper: when I handed the AI the result summary and asked it to *interpret* the numbers, it produced five distinct misinterpretations — reading the single worst-case max (51 ms) as systemic degradation, mistaking one process at 42% CPU (on a 10-core box) for a saturated breaking point, and reporting a think-time-limited baseline (12.8 req/s) as the server’s maximum capacity when Stress had already sustained 151 req/s — and it prescribed SQLite optimizations (an index for a leading-wildcard `LIKE`, a connection pool for an embedded single-file DB) that don’t apply to the actual driver. Every one of these was caught only by reading the source, probing the live app, and re-computing the metrics from the raw `.jtl`. My recommendation: use AI to produce the first draft and the tedious plumbing, but treat every parameter, metric, and assumption as unverified until confirmed against the code, the seed data, and a real run — the human is responsible for that verification.

6. Mandatory Disclosure (paste verbatim)

The test plans, CSV data, monitoring/analysis scripts, and evidence documents were initially generated by Claude Code; I reviewed and corrected them (keyword fix, lockout behaviour, timer placement, listener choice, content assertions), executed all four scenarios myself on my own hardware, and wrote Section 5 and the human-review reasoning entirely by myself. For Task 2, the AI produced the performance analysis and the optimization proposals; the misinterpretation hunt (the 5 corrections against the raw `.jtl`) and the feasible/hallucinated verdict on each optimization are my own work. The detailed AI Audit Report is this appendix. I confirm I did not use AI to generate any artifact listed in the prohibited category (raw `.jtl` logs, resource-monitor screenshots, the demo video, or the hardware report).

Signature

Student name (printed):	DANG TRUONG NGUYEN
Student ID:	23127438
Class / Cohort:	23KTPM3
Course:	CS423 / CSC13003 – Software Testing
Instructor:	Msc. Tran Thi Bich Hanh
Date:	15/08/2026
Signature:	

References

- Kharbach, M. (2026). AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0.

- ISTQB Foundation Level Syllabus (latest version).
- Hardman, P. (2025). A Post-AI Learning Taxonomy.
- Fuster Rabella, M. (2025). OECD Education Working Paper No. 338.
- Perkins, M., Roe, J., & Furze, L. (2025). AI Assessment Scale.
- Anthropic (2025). Building reliable AI test agents — engineering blog.
- DeepEval & Promptfoo documentation — testing frameworks for LLM systems.